

Queen Mary University of London
School of Engineering and Materials Science

EMS511U
Robot Design and Mechatronics Movement

First name : Teligama

Last name : Gunasekera

QMUL Student ID number: 230643507

Team Design : G

Group : 13

Team members :

Name	Student ID
Jasvene Ahilan	231172745
Keesha Sia	230502916

Two Degree of Freedom Robotic Manipulator

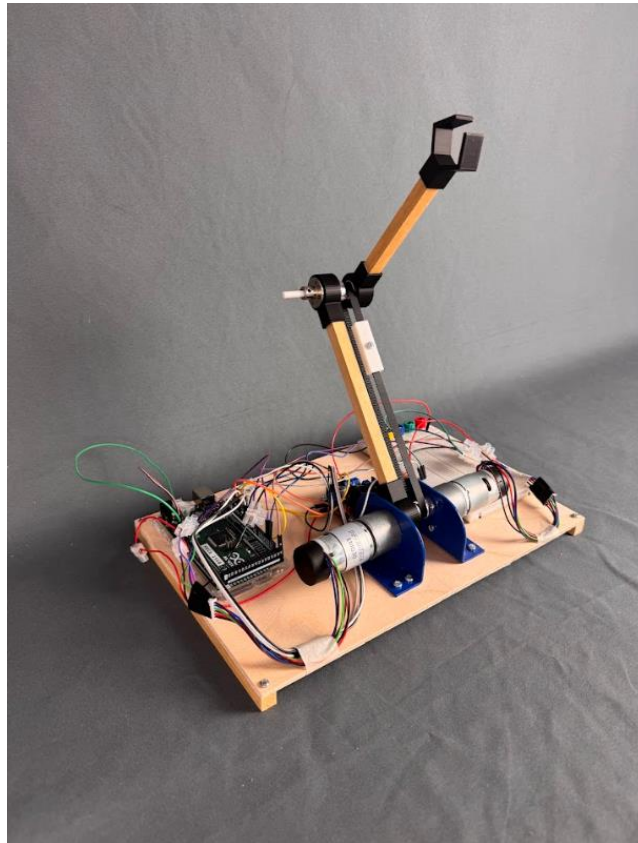


Figure 1

Abstract

This report presents the process and outcomes of developing a 2 Degree of Freedom (2-DOF) robotic arm as part of the EMS511U Robot Design and Mechatronics module. The robotic arm was designed using CAD modeling, 3D printing, and lab-fabricated components. It was controlled using PID controllers implemented in MATLAB Simulink, running on an Arduino Mega 2560 micro controller. The project involved mechanical design, electronics integration, control algorithm development, and testing. The personal experience in this project was enriching and collaborative. I primarily contributed to the control system design, simulation setup, and electronics integration. I would confirm that all members contributed equally. Jasvene focused heavily on assembly and manufacturing while Keesha handled a majority of the CAD modelling and 3D printing. Overall, the project was valuable team-based engineering experience.

Table of Contents

1. Introduction	3
1.1. Overview of the Project	3
1.2. Objective	3
2. Materials and methods	4
2.1. Forward and Inverse Kinematics	4
2.2. DH Kinematics	5
2.3. Design Sketches	6
2.4. CAD Design	7
2.5. Electronics and Electromechanics Components Used	10
2.6. Manufacturing and Assembly Process	11
2.7. Control System Architecture	12
3. Results	14
4. Discussion and Conclusion	16
5. References	17
6. Appendix	18

1. Introduction

1.1. Overview of the Project

Robotic manipulators are widely used in automation, medical robotics, and manufacturing processes. This project aimed to develop a simplified 2 DOF planar robotic arm that mimics basic pick and place functionality. The Design 'G' 2 DOF configuration (Figure 2) was chosen due to its straightforward kinematic properties, which made it ideal for applying theoretical principles such as forward and inverse kinematics, Denavit Hartenberg (DH) modeling, and PID based control. The design's both degrees of freedom are rotational in a 2D environment. So that, the robot's end effector's position can be presented on a x, y plane (Figure 3).

1.2. Objective

The main objective of this project is to design, simulate, fabricate, and test a simple two degree of freedom robotic arm which is controlled by a PID controller using available lab resources and open-source control tools.

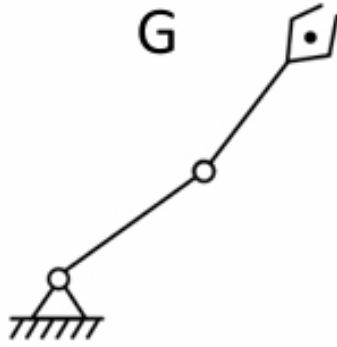


Figure 2 : 2 DOF Configuration

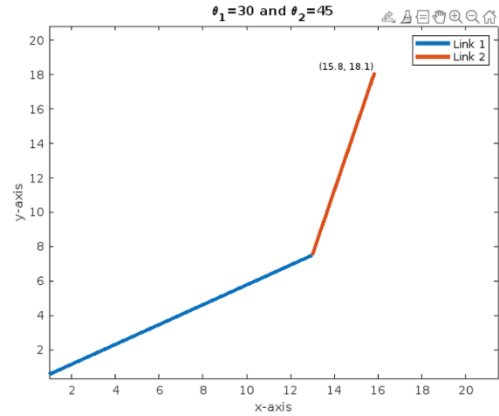
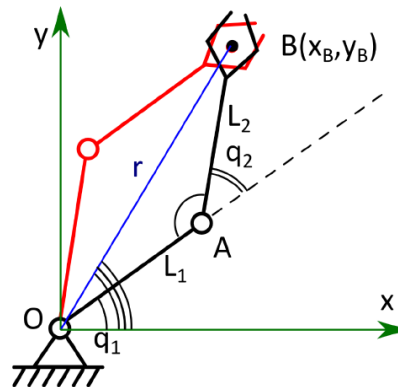


Figure 3 : End effector's position

2. Materials and Methods

2.1. Forward and Inverse Kinematics



Forward kinematics is the process of calculating the position and orientation of the end effector of the robot manipulator based on the known values of its joint variables such as angles for rotational joints, displacements for prismatic joints and the geometric parameters of its links.

$$\begin{aligned} x_B &= L_1 \cos q_1 + L_2 \cos(q_1 + q_2) \\ y_B &= L_1 \sin q_1 + L_2 \sin(q_1 + q_2) \end{aligned}$$

Figure 4 : Forward kinematics

Inverse kinematics tackles the opposite of the forward kinematics. So that, it is used to calculate the joint angles in order to achieve a desired position and orientation of the robot's end effector.

$$\begin{aligned}
q_2 &= \tan^{-1} \frac{\pm \sqrt{1-D^2}}{D} \\
q_1 &= \tan^{-1} \frac{y_B}{x_B} - \tan^{-1} \frac{L_2 \sin q_2}{L_1 + L_2 \cos q_2} \\
D &\equiv \frac{x_B^2 + y_B^2 - L_1^2 - L_2^2}{2L_1 L_2}
\end{aligned}$$

Figure 5 : Inverse kinematics

2.2. DH Kinematics

Denavit-Hartenberg kinematics is a systematic and standardized method used to describe the geometry and kinematics of robotic manipulators. It provides a consistent way to establish coordinate frames for each link of a robot and to derive the transformations between these frames.

Notations:

Joint Number : i

Link Offset : d_i

Link Length : a_i

Joint Angle : θ_i

Link Twist : α_i

For our Planar 2-Link Robot Arm :

L	a_i	α_i	d_i	θ_i
1	a_1	0	0	θ_1
2	a_2	0	0	θ_2

$$\begin{aligned}
A_i &= R_{z,\theta_i} \text{Trans}_{z,d_i} \text{Trans}_{x,a_i} R_{x,\alpha_i} \quad (3.10) \\
&= \begin{bmatrix} c\theta_i & -s\theta_i & 0 & 0 \\ s\theta_i & c\theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c\alpha_i & -s\alpha_i & 0 \\ 0 & s\alpha_i & c\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}$$

$$A_1 = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 & a_1\cos\theta_1 \\ \sin\theta_1 & \cos\theta_1 & 0 & a_1\sin\theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & a_2\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & 0 & a_2\sin\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Homogenous Transformation Matrix :

$$T_2^0 = A_1 A_2 = \begin{bmatrix} \cos\theta_1 \cos\theta_2 & -\sin\theta_1 \sin\theta_2 & 0 & a_1\cos\theta_1 + a_2\cos\theta_2 \\ \sin\theta_1 \cos\theta_2 & \cos\theta_1 \sin\theta_2 & 0 & a_1\sin\theta_1 + a_2\sin\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$x = a_1\cos\theta_1 + a_2\cos\theta_2$$

$$y = a_1\sin\theta_1 + a_2\sin\theta_2$$

2.3. Design Sketches

The design sketches helped identify part shapes, required tolerances, and integration points. Initial design sketches involved two rotational joints, two arms and a simple gripper as well as the 2nd motor was placed at the end of the first arm (Figure 6).

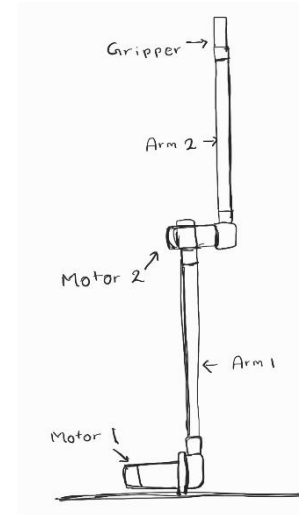
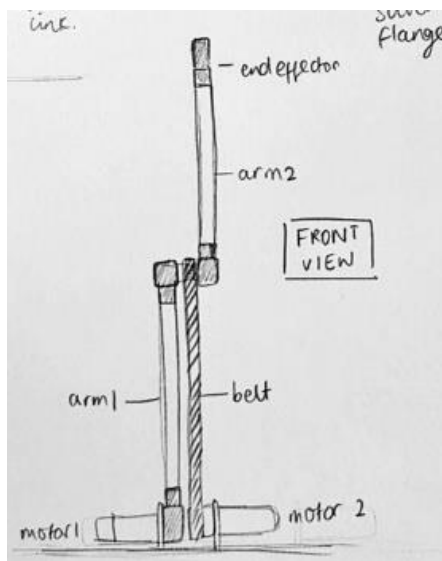


Figure 6 : Initial Design Sketches



The selected design includes two rotational joints, two arms, a simple gripper, a belt and pulley. As the final design, belt driven mechanism has been added instead of placing a motor on top of a arm. So that, it improves the stability, durability and the efficiency of the manipulator.

Figure 7 : Final Design Sketches

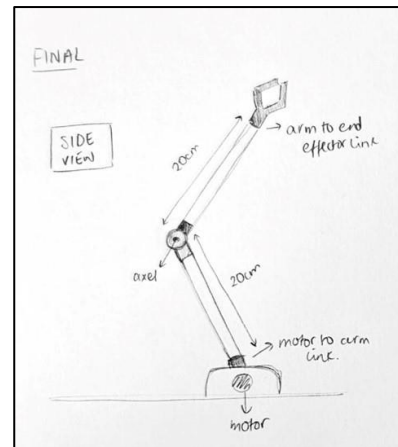
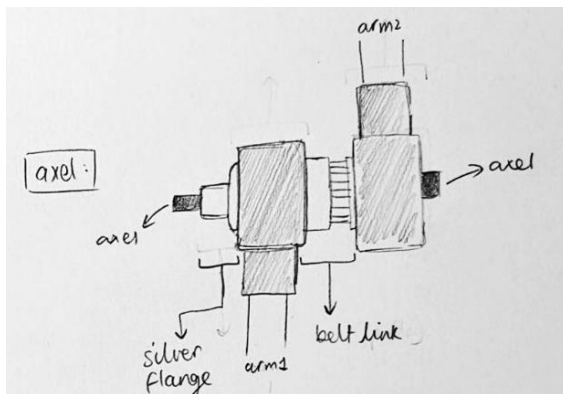


Figure 8 : Final Design Sketches

2.4. CAD Design

CAD modeling was done using Onshape and Solidworks. Joints, the axle, and the gripper were custom designed while flange, belt, pulley and motors were designed according to the given dimensions. A full assembly model ensured all parts fit together.

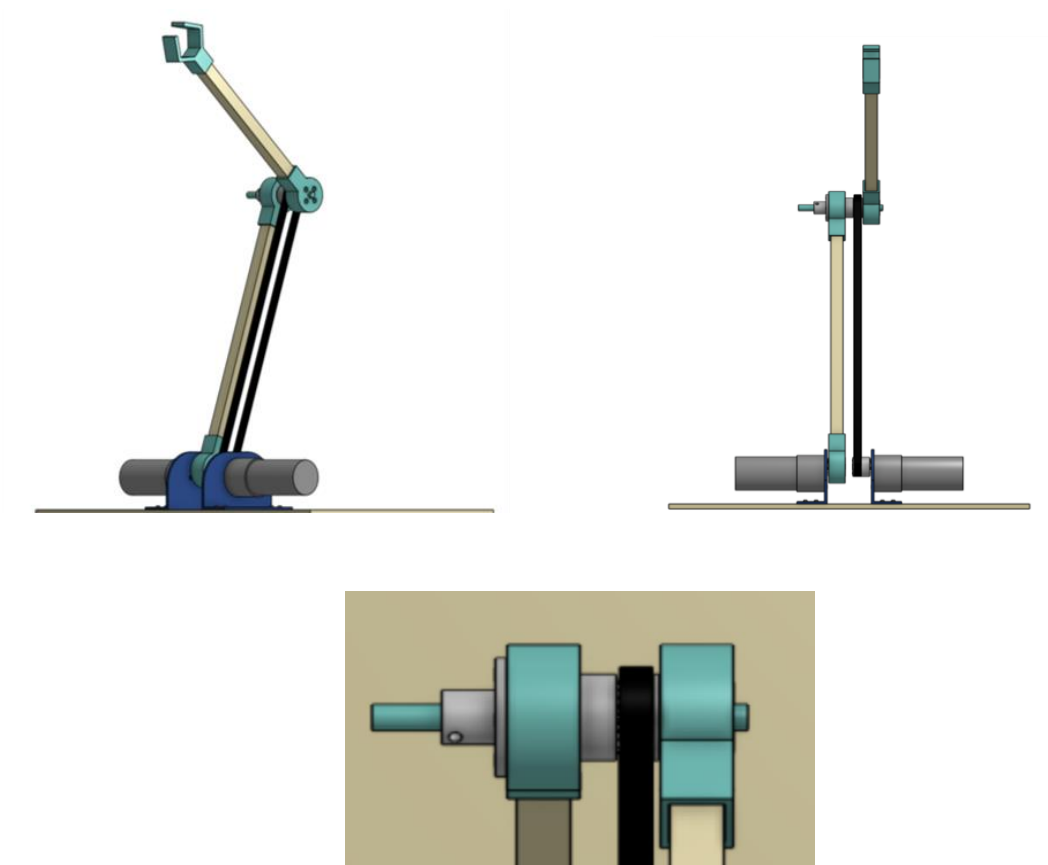


Figure 9 : Assembly CAD

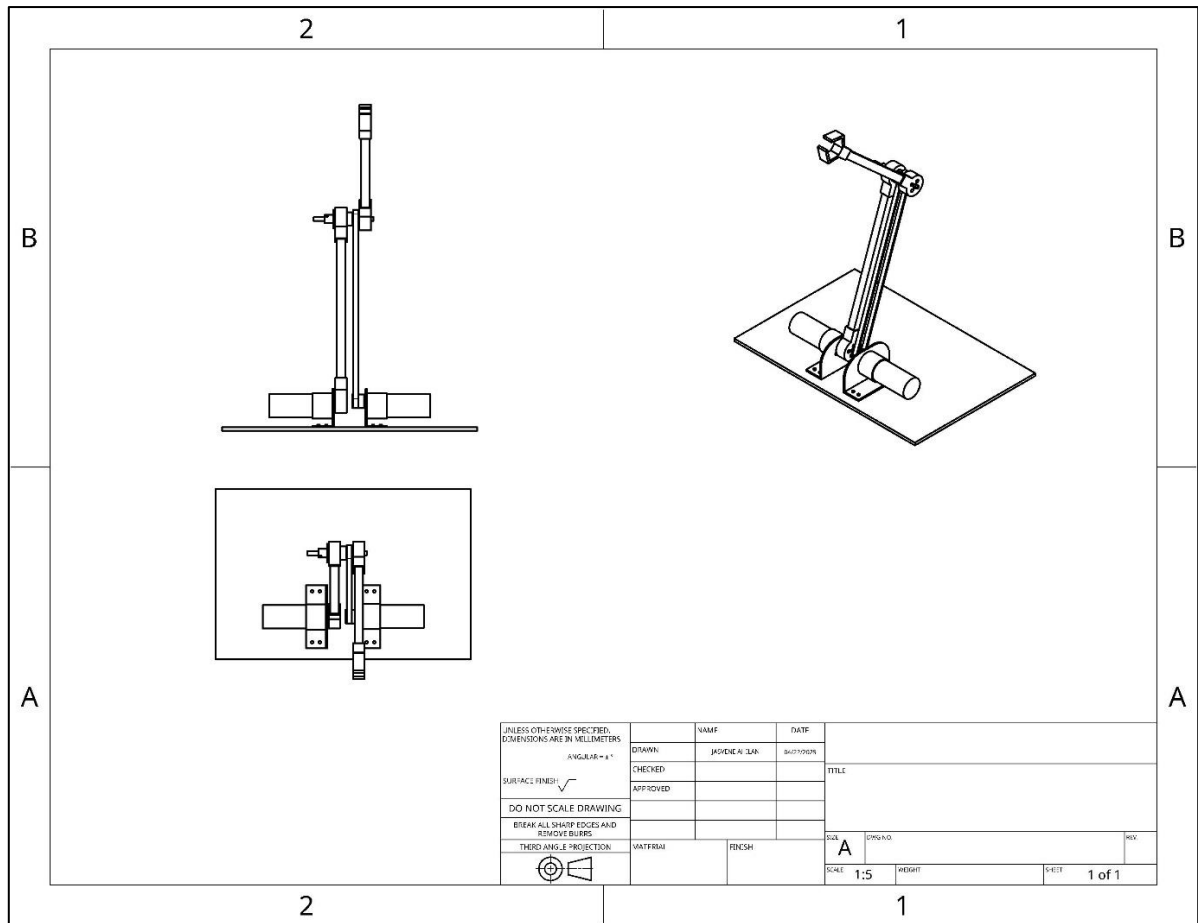


Figure 10 : Engineering Drawing of the Assembly

Among the designed components in CAD, these two components were designed by me and they were used to 3D print later.

- Part 1 : Gripper – Includes inner channels for gripping and square hole to fit onto the arm (Figure 11).
- Part 2 : Motor Joint – Custom part designed to fit to the EMG 30 motor and the arm securely together and transfer motion to the arm 1 (Figure 12).

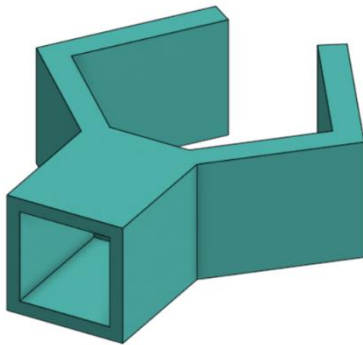


Figure 11 : Part 1 - Gripper

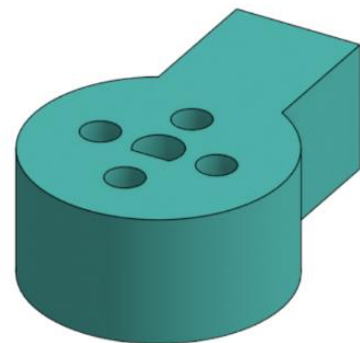


Figure 12 : Part 2 – Motor Joint

Engineering drawings of each part represent detailed specifications with visual representations.

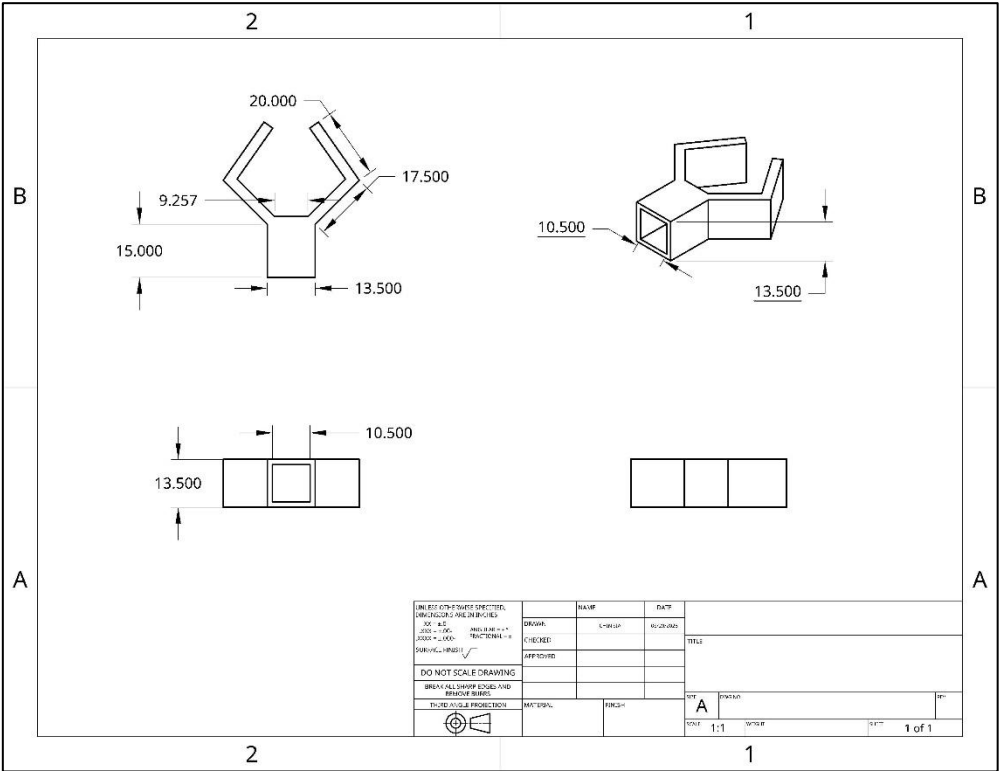


Figure 13 : Engineering Drawing – Part 1

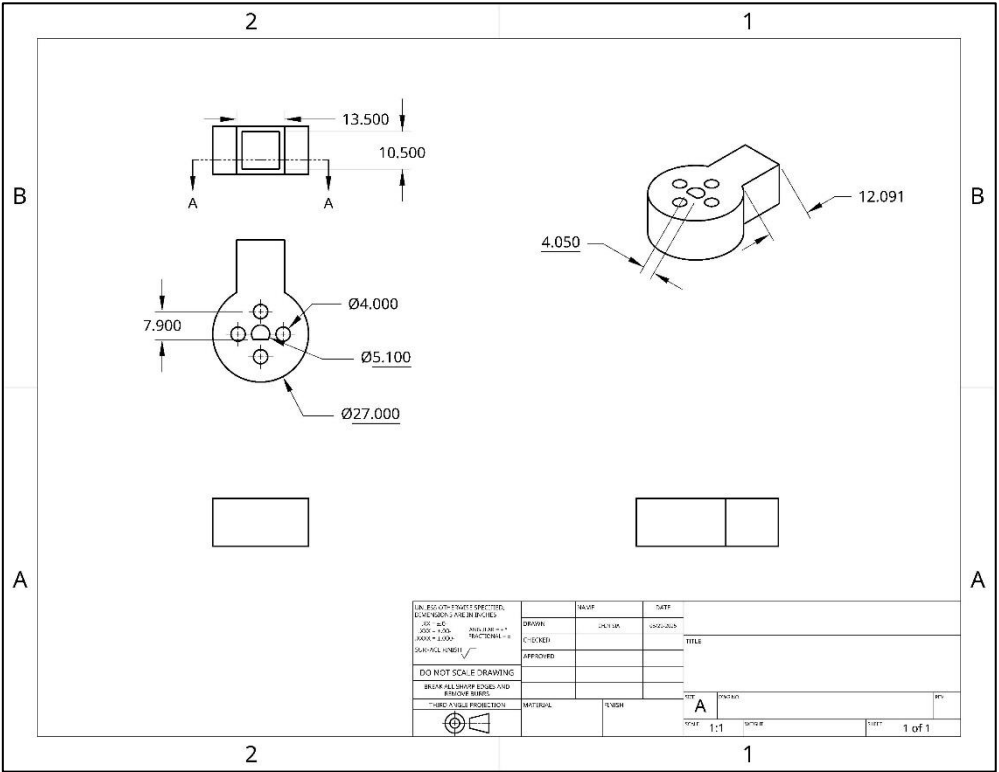


Figure 14 : Engineering Drawing – Part 2

2.5. Electronics and Electromechanics Components Used

- Arduino Mega 2560

The Arduino Mega 2560 acts as the brain of the robotic arm. It receives desired joint angles (θ_1 and θ_2) from MATLAB Simulink, sends PWM (Pulse Width Modulation) signals to the motor driver to control the motors and reads encoder signals from the motors to measure their actual positions.

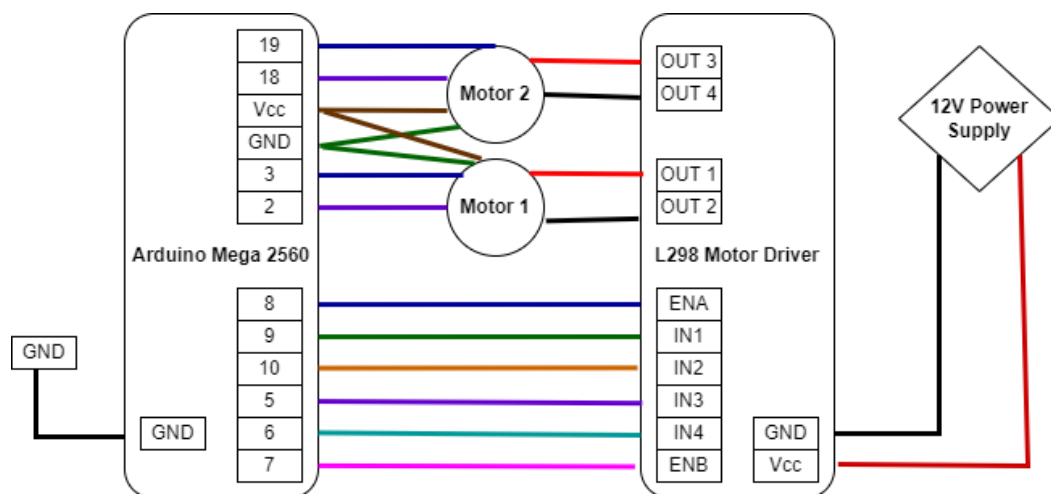
- EMG 30 DC motors with encoders

Motors provide rotational actuation to each joint while encoders measure the rotation angle and direction, sending pulses back to the Arduino.

- L298N motor driver

This L298N module powered by an external 12V supply controls both motors and accepts control signals from the Arduino. It can drive the motors in both directions with variable speed.

Integration of electronics component as a schematic diagram :



- GT20 belt and pulleys

This mechanical system is used to transmit rotational motion from the motor shaft to the 2nd robot arm. It provides torque transmission and rotational flexibility between joints and helps to reduce backlash and increase accuracy when properly tensioned.

As the Integration, Pulleys were mounted on the motor shaft and the joint axle while the GT20 belt linked them, ensuring synchronized motion. Further, proper tension was required to avoid slip and vibration.

2.6. Manufacturing and Assembly Process

As the first step of manufacturing, Custom joints, grippers and a axel were manufactured using the CAD model and 3D printing (Figure 15). Next, two wooden arms which dimensions are 1cmx1cmx15cm and 1cmx1cmx11cm were fabricated using lab tools. Finally, 0.4cmx20cmx30cm wooden sheet has been manufactured to use as the base of the robot. Electronics and other components such as belt and pulleys, flanges, bearings, nuts and bolts were supplied by the lab.



Figure 15 : 3D Printed Parts

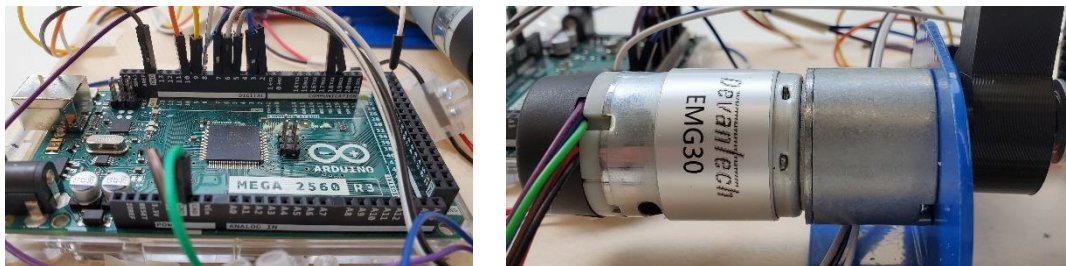


Figure 16 : Arduino Mega 2560 & EMG 30 Motor

Assembly included mounting motors to the base, attaching arm to the motor using a joint and aligning pulleys. A pulley, an axel, arm 1 and arm 2 with the gripper were assembled as shown on figure 17 . Multiple reprints of custom components were necessary to refine fitting.

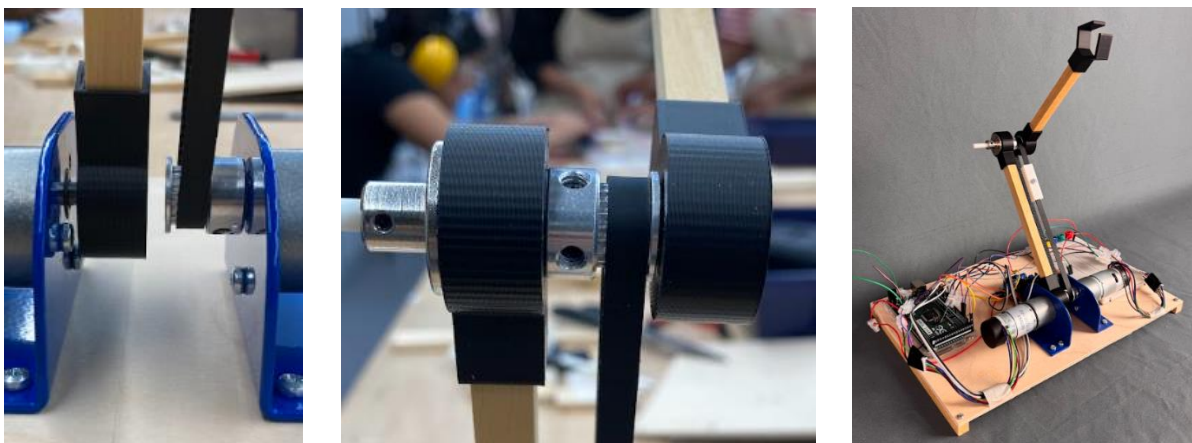


Figure 17 : Final Assembly

2.7. Control System Architecture

Simulink was used to design and tune the PID controllers for each joint. Input was set and feedback from encoders was used to adjust motor output via PWM. Main control system architecture includes a few subsystems which are kinematics, PID controller, Motor controller and Encoder for each motor. Input is set as a coordinate of the end effector's target position and then output is to reach to the point. Further, PID controlled Arduino based program has been developed as well because of the experienced compiling errors in Simulink (Arduino PID program has been added in the appendix section).

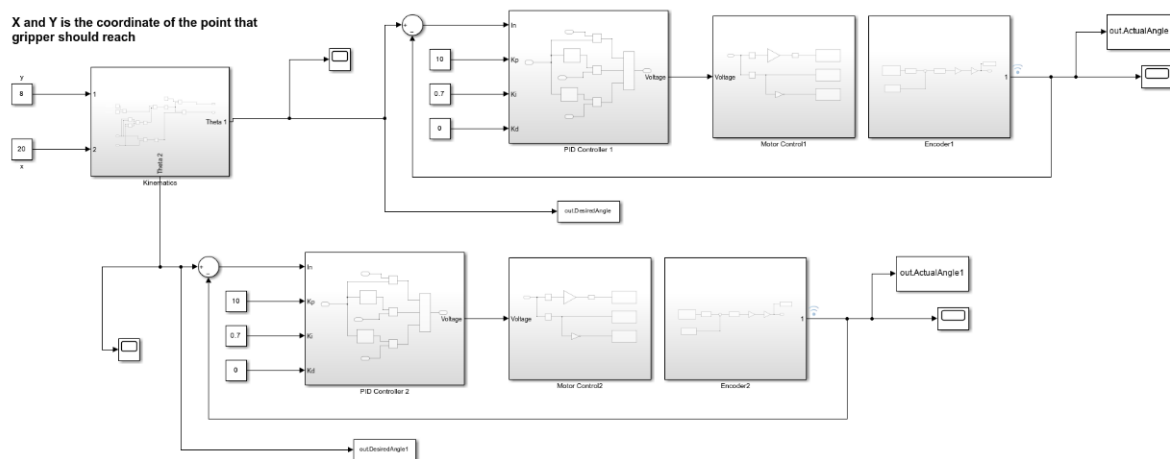


Figure 18 : Main Simulink Control System

Kinematics Subsystem :

In the Kinematics subsystem, it converts system input to two angles which are theta 1 and theta 2 using inverse kinematics.

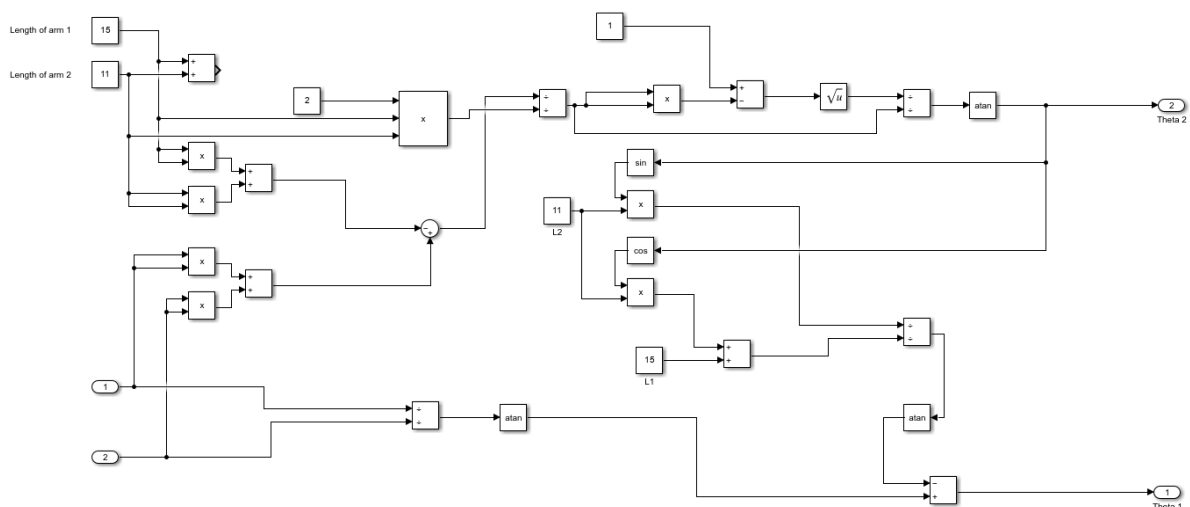


Figure 19 : Kinematics Subsystem

PID Controller Subsystem :

This subsystem is used to tune the PID to achieve optimized results and there are two PID controller subsystems to tune each motor output. A PID controller is a feedback control loop mechanism widely used in industrial control systems requiring continuously modulated control. It's designed to minimize the error between a desired setpoint and a measured process variable by adjusting a control output.

When tuning the PID, higher Proportional term (K_p) and Integral term (K_i) values result in a relatively quick response and reduced steady state error, but they can also lead to significant overshoot. However, by adjusting Derivative term (K_d) value, damping can be improved so that it leads to less overshoot.

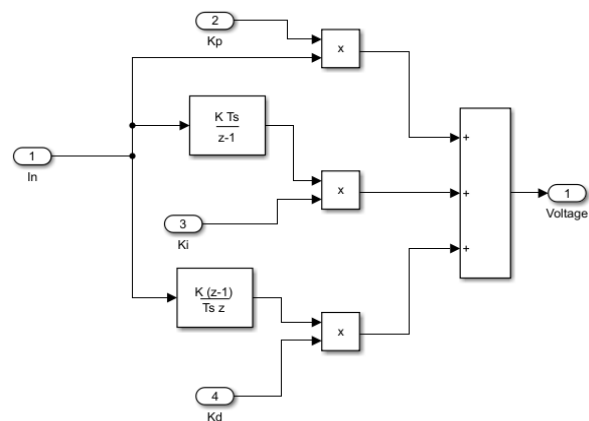
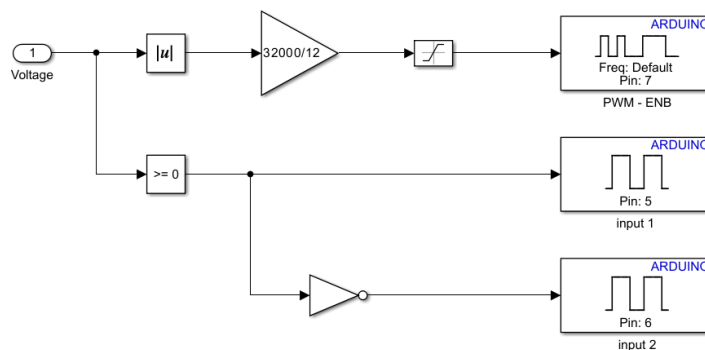


Figure 20 : PID Controller Subsystem

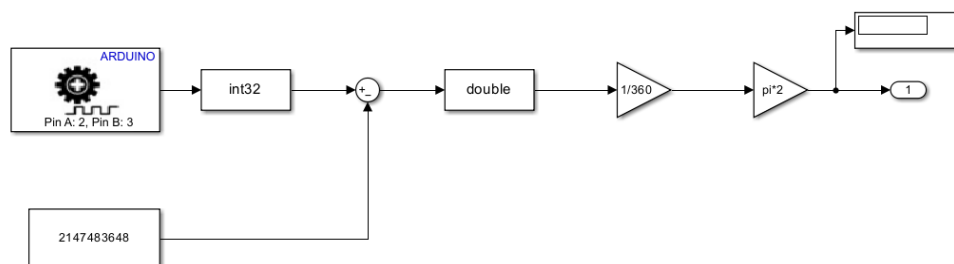
Motor Control Subsystem :

When the PID controller sends relevant voltage to the motor controller, it drives the motor for appropriate direction with the speed.



Encoder Subsystem :

Encoder subsystems take the position of the arm using the encoder of the motor and send it to calculate the error between target angle and reached angle.



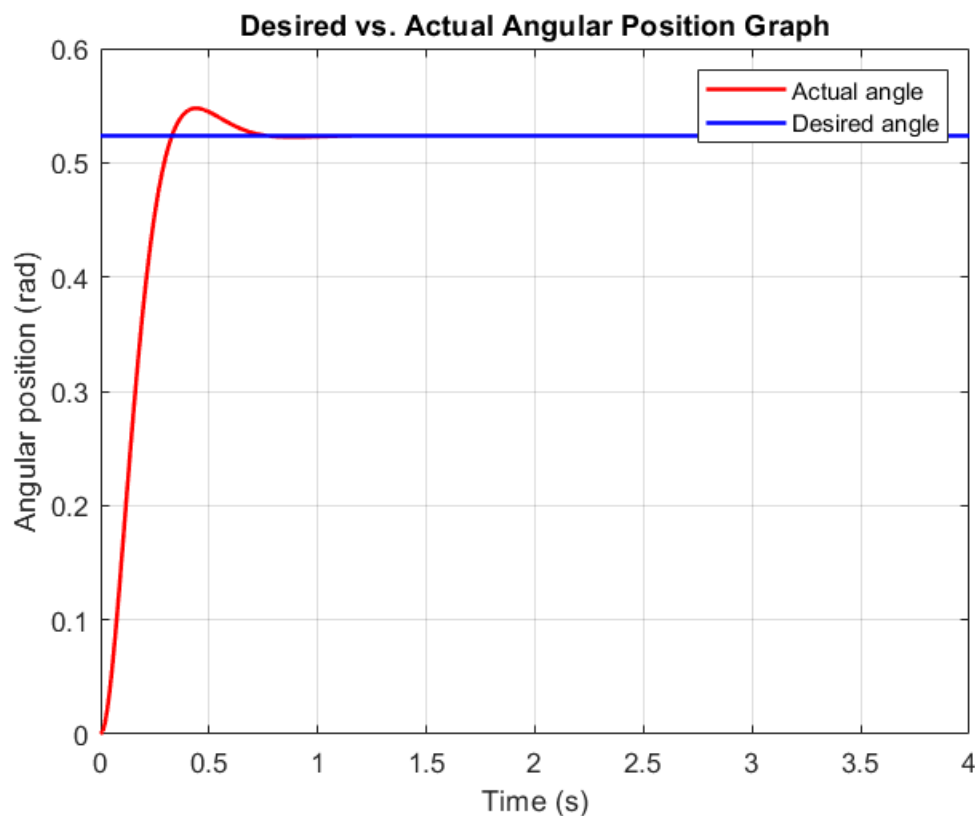
3. Results

Testing showed that the arm could track joint angle commands using PID control. After implementing the set of tests, PID values for each motor have been determined to obtain optimized results. As the target of PID tuning, faster response, less overshoot and less steady state error have been expected. So that, the table below (Table 1) represents the finalized PID values for the motor 2.

	Proportional Gain (K_p)	Integral Gain (K_i)	Derivative Gain (K_d)
Value	1	0.7	0

Table 1 : PID Values of motor 2

The performance plot indicated some overshoot initially but were improved through tuning. Feedback from encoders was occasionally noisy, but adequate for closed-loop control. Plots of setpoint vs actual angle for motor 2 are shown on Graph 1.



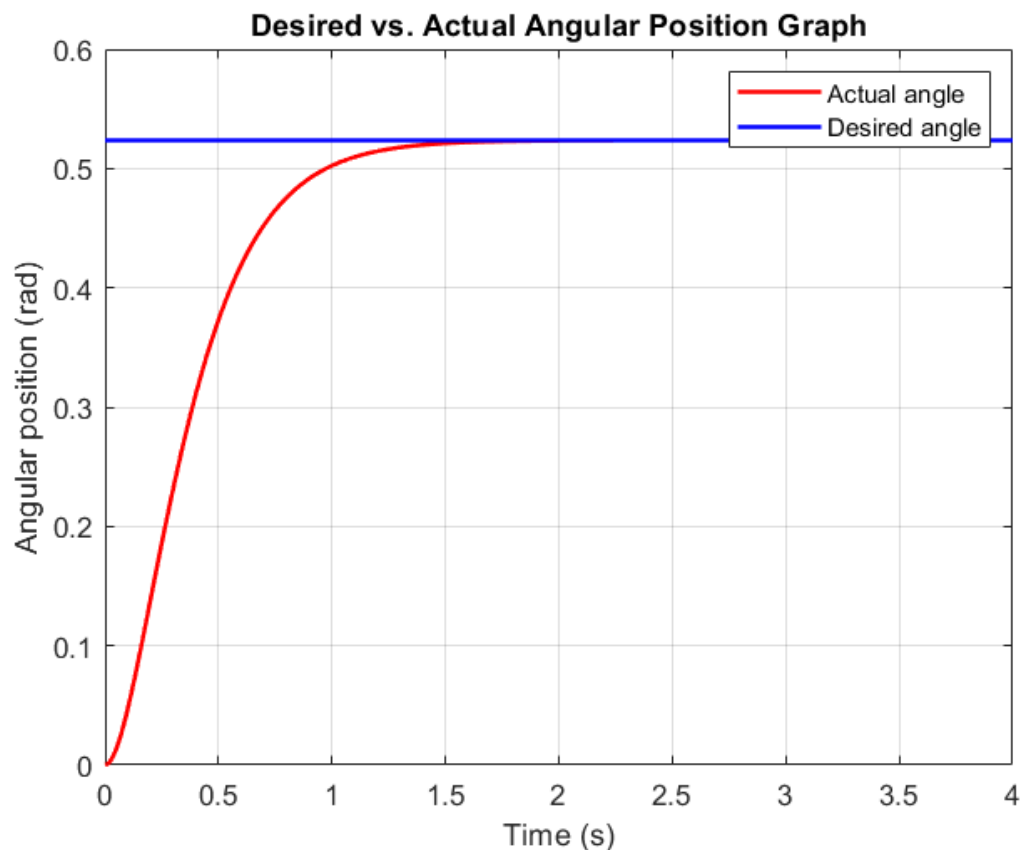
Graph 1 : Feedback graph of the motor 2

However, determining a particular PID values for motor 1 became a considerable challenge. The main reason for this is the effect of gravity. Unlike arm 2 and motor 2, motor 1 moves the whole unit includes two arms, axels and pulleys so that at some points, the gravity of the unit heavily effects as a resistance for the motor while at some points, it supports the motion of the arm.

If PID values are set to perform as targeted initially with the resistance of gravity, it moves with too low rising time with higher overshoot which made the robot unstable when the gravity doesn't affect the motion. Therefore, PID values for motor 1 were set to similar to the motor 2. So that, it performs as shown in Graph 1 when gravity doesn't affect and the rest of time, it performs higher rising time with no overshoot and steady state error as shown in Graph 2.

	Proportional Gain (K_p)	Integral Gain (K_i)	Derivative Gain (K_d)
Value	1	0.7	0

Table 2 : PID Values of motor 1



Graph 2 : Feedback graph of the motor 1

4. Discussion and Conclusions

The development of the 2-DOF robotic arm presented a comprehensive learning opportunity, combining mechanical design, electronics integration, and real-time control system implementation. The project's primary objective of constructing and controlling a planar robot arm using PID control was successfully achieved, albeit with several technical challenges along the way.

From a mechanical standpoint, the design of the arm using a combination of wooden bars and 3D printed joints offered both flexibility and customizability. However, the use of different materials led to integration difficulties, particularly when it came to aligning parts with precise tolerances. The GT20 timing belt and pulley system were effective in transmitting torque but introduced mechanical stress on unsupported axles. This caused slight misalignments that negatively affected the accuracy of the encoder feedback and introduced vibration during motion.

On the electrical side, the integration of EMG 30 DC motors with encoders and the L298N motor driver provided a robust foundation for feedback-controlled motion. The Arduino Mega 2560 served as a reliable microcontroller platform, facilitating seamless communication between the MATLAB Simulink environment and the hardware components. Nevertheless, the encoder feedback was occasionally unstable due to mechanical misalignment and loose connections, requiring multiple rounds of troubleshooting.

Control wise, the implementation of PID controllers in Simulink allowed us to precisely regulate the angular position of each motor. The PID control structure was successfully tuned to achieve desired position tracking with minimal steady state error and acceptable transient response. Still, the controllers sometimes struggled to compensate for load changes and vibrations especially when the center of gravity shifted during arm movement. Furthermore, too many bugs and compiling errors have been experienced in the Simulink, and it made Simulink PID controller unusable.

However, in order to address the mentioned challenges, Multiple iterations of CAD parts were printed and tested to refine the fit and function of each joint. Moreover, Arduino IDE was used to design a PID controller instead of using Simulink to avoid compiling errors. Although a one-sided axle support was initially used and the need for dual support in future designs to reduce flex and vibration and improve PID performance has been suggested. Replacing wooden arms with aluminum or carbon fiber components would improve structural rigidity and reduce vibration. Adding a servo-controlled gripper would increase the robot's functionality for real-world tasks such as object manipulation. Increasing the base height to route cables cleanly underneath the frame would reduce clutter and risk of disconnection.

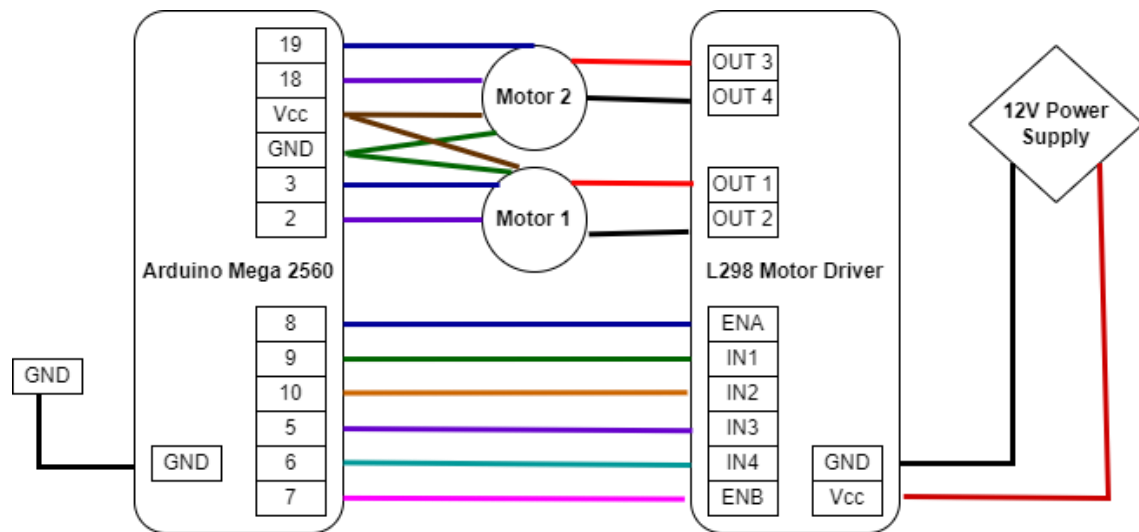
In conclusion, this project provided valuable hands-on experience in robotic system design and highlighted the importance of iterative testing and interdisciplinary collaboration. The skills and knowledge acquired during this project lay a strong foundation for future work in automation, robotics, and control systems.

5. References

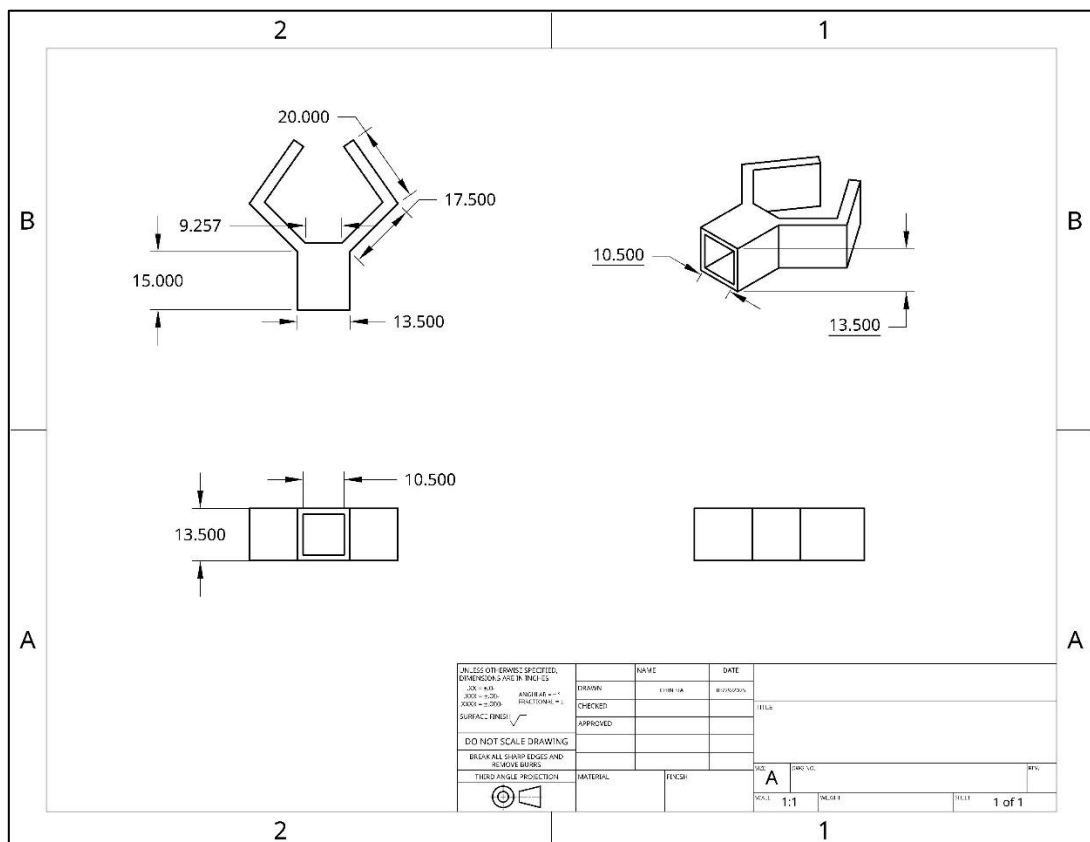
1. *Dr Ildar Farkhatdinov – Lecture 1 Introduction to the course Mechanical design Mechatronics components, 2024.*
2. *Dr Ildar Farkhatdinov – Review of Robot Kinematics, 2024.*
3. *Dr Ildar Farkhatdinov – PID control: discrete implementation, 2024.*
4. *Dr Ildar Farkhatdinov – DC motor dynamics and position control, 2025.*
5. *Dr Ketao Zhang – Robot Design and Mechatronics Laboratory Document 1, 2025.*
6. *Dr Ketao Zhang – Robot Design and Mechatronics Laboratory Document 2, 2025.*
7. *Dr Ketao Zhang – Robot Design and Mechatronics Laboratory Document 3, 2025.*
8. *Dr Ketao Zhang – Robot Design and Mechatronics Laboratory Document 4, 2025.*
9. *Dr Ketao Zhang – Robot Design and Mechatronics Laboratory Document 5, 2025.*
10. *DroneBot Workshop Tutorial – Controlling DC Motors with the L298N Dual H-Bridge, 2021.*
11. *MathWorks – [PID Controller Design in Simulink - MATLAB & Simulink](#), 2012.*
12. *Terry Denery – [Engineers Studio #30 -- Introduction to Controlling DC Motor with Arduino](#), 2023.*
13. *Donghwa.Ryu – [DC Motor Control | Part2 - Arduino using Matlab Simulink](#), 2024.*
14. *Donghwa.Ryu – [Stepper Motor Control | Part1 \(Full Step / Half Step Control\)- Arduino Using Matlab Simulink](#), 2024.*
15. *Donghwa.Ryu – [DC Motor Control \(PID Speed Control\) | Part3 - Arduino Using Matlab Simulink](#), 2024.*

6. Appendix

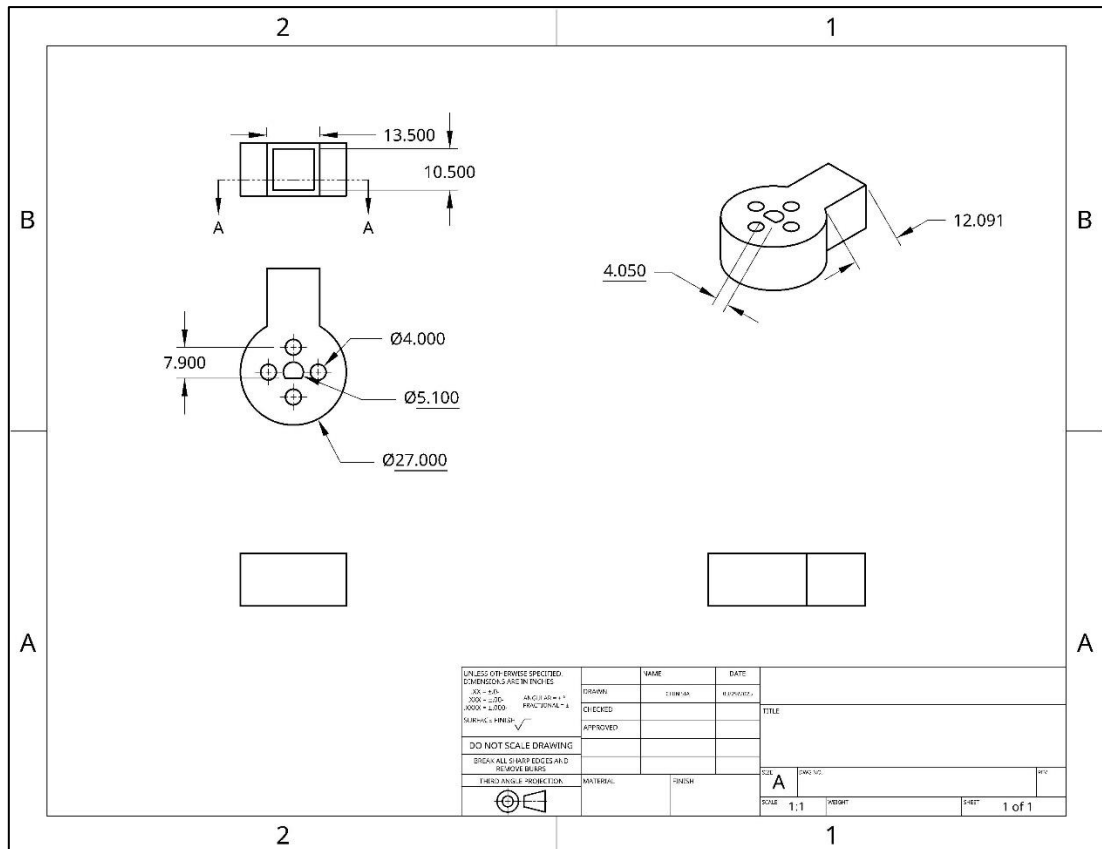
Electrical Schematic Diagram :



Professional CAD Drawings :



Engineering Drawing – Part 1



Bill of Materials :

Component	Cost	Link
Arduino Mega 2560	£43.39	Arduino Mega 2560 REV3 [A000067] – ATmega2560, 16MHz, 54 Digital I/O, 16 Analog In, 256KB Flash, USB, Arduino IDE Compatible for Advanced Projects & Prototyping : Amazon.co.uk: Computers & Accessories
EMG 30 DC motors (x2)	£32.84 x 2 = £65.68	EMG30 - GearMotor with encoder
L298 Dual H bridge Motor Driver	£3.99	RKL298 L298 H-Bridge Motor Drive Project PCB Self Build Kit
Wires	2m, 11 colours: £5.95	1.4A (7/0.2mm) Multi Strand Wire Pack - 2m x 11 Colours - Small Scale Lights
GT20 Belt and Pulley	Pack of 8: £9.99 Per 1: £1.25	Dadabig 26 PCS 3D Printer Timing Belt Kit, GT2 20 Teeth Timing Pulley Wheel 5mm + 5M GT2 Timing Belt 6mm Width with Allen Wrench for 3D Printer: Amazon.co.uk: Business, Industry & Science

<i>Metal Flange</i>	2 pieces : £7.99 1 piece : £3.99	<u>CHACNS Flange Shaft Coupling with Screws Outer Diameter 16mm Inner Diameter 12mm Flange Coupling Connector 2PCS : Amazon.co.uk: Business, Industry & Science</u>
<i>Motor Bracket (x2)</i>	£4.38 x 2 = £8.76	<u>EMG30 mounting bracket</u>
<i>Bearing</i>	10 pieces : £6.96 1 piece : £0.70	<u>Othmro 10pcs 625ZZ Bearings, Deep Groove Ball Bearing Double Shield 5mm x 16mm x 5mm, High Carbon Steel GCr15 625 Bearing : Amazon.co.uk: Business, Industry & Science</u>
<i>Wooden Arms</i>	50 pieces : £12.99 2 piece : £0.52	<u>BUYGOO 50Pcs Wooden Sticks for Craft 25cm Square Wooden Dowels Rods Natural 1cm Wood Blocks for Carving Craft DIY(25 x 1 x 1cm) : Amazon.co.uk: DIY & Tools</u>
<i>3D Printed End-Effector</i>	0.018 GBP per gram 4.41 x 0.018 = £0.079	
<i>3D Printed Axel</i>	0.018 GBP per gram 1.53 x 0.018 = £0.028	
<i>3D Printed Motor Joint</i>	0.018 GBP per gram 5.44 x 0.018= £0.098	
<i>3D Printed Bearing Joint</i>	0.018 GBP per gram 10.61x 0.018 = £0.191	
	Total = £134. 63	